

309. Best Time to Buy And Sell Stock With Cooldown - Explanation

[Problem Link](#)

Description

You are given an integer array `prices` where `prices[i]` is the price of NeetCode on the `i`th day.

You may buy and sell one NeetCode multiple times with the following restrictions:

- After you sell your NeetCode, you cannot buy another one on the next day (i.e., there is a cooldown period of one day).
- You may only own at most one NeetCode at a time.

You may complete as many transactions as you like.

Return the **maximum profit** you can achieve.

Example 1:

Input: `prices = [1,3,4,0,4]`

Output: 6

Explanation: Buy on day 0 (price = 1) and sell on day 1 (price = 3), profit = $3 - 1 = 2$. Then buy on day 3 (price = 0) and sell on day 4 (price = 4), profit = $4 - 0 = 4$. Total profit is $2 + 4 = 6$.

Example 2:

Input: prices = [1]

Output: 0

Constraints:

- $1 \leq \text{prices.length} \leq 5000$
- $0 \leq \text{prices}[i] \leq 1000$

Topics

Recommended Time & Space Complexity

You should aim for a solution as good or better than $O(n)$ time and $O(1)$ space, where n is the size of the input array.

Hint 1

Try to think in terms of recursion and visualize it as a decision tree. Can you determine the possible decisions at each recursion step? Also, can you

identify the base cases and the essential information that needs to be tracked during recursion?

Hint 2

At each recursion step, we can buy only if we haven't already bought a coin, or we can sell if we own one. When buying, we subtract the coin value, and when selling, we add it. We explore all possible buying and selling options recursively, iterating through the coins from left to right using index `i`. For the cooldown condition, if we buy a coin, we increment the index `i` by two.

Hint 3

We can use a boolean variable `canBuy` to indicate whether buying is allowed at the current recursive step. If we go out of bounds, we return `0`. This approach is exponential. Can you think of a way to optimize it?

Hint 4

We can use memoization to cache the results of recursive calls and avoid recalculations. A hash map or a `2D` array can be used to store these results.

Company Tags

Please upgrade to NeetCode Pro to view company tags.

Prerequisites

Before attempting this problem, you should be comfortable with:

- **Recursion** - Making decisions at each step and exploring all possibilities
- **Dynamic Programming** - Memoization to cache results and avoid redundant calculations
- **State Machine Thinking** - Tracking multiple states (buying vs. selling) throughout the problem
- **Space Optimization** - Reducing DP table to constant space using rolling variables

1. Recursion

Intuition

This problem is about deciding the best days to buy and sell a stock to maximize profit, with one important rule: **after selling a stock, you must wait one day before buying again (cooldown)**.

At every day, we have two possible states:

- we are **allowed to buy** (we are not holding a stock)
- we are **allowed to sell** (we are currently holding a stock)

Using recursion, we try **all possible decisions** starting from day `0` and choose the one that gives the maximum profit.

At each step, we either:

- take an action (buy or sell), or
- skip the day (cooldown)

The recursive function represents:

"What is the maximum profit we can make starting from day `i`, given whether we are allowed to buy or not?"

Algorithm

1. Define a recursive function `dfs(i, buying)`:
 - `i` represents the current day
 - `buying` indicates whether we are allowed to buy (`true`) or must sell (`false`)
2. If `i` goes beyond the last day:
 - Return `0` because no more profit can be made
3. Always compute the option to **skip the current day** (cooldown):
 - Move to the next day without changing state
4. If we are allowed to buy:
 - Option 1: Buy the stock today (subtract price and move to selling state)
 - Option 2: Skip the day
 - Take the maximum of these two options
5. If we are holding a stock:
 - Option 1: Sell the stock today (add price and skip the next day due to cooldown)
 - Option 2: Skip the day
 - Take the maximum of these two options
6. Start the recursion from day `0` with `buying = true`

7. Return the result of this initial call

Best Time to Buy and Sell Stock with Cooldown - Dynamic Programming

Step 1 / 18

Prices

1

0

2

1

3

2

0

3

2

4

Call Stack

bottom

dfs(0, True)

top

Start with prices=[1,2,3,0,2] and call dfs(0, True) to find max profit starting from day 0 with ability to buy.

Upgrade to Pro to unlock visualizations

0.5x1x2x

```
public class Solution {  
    public int maxProfit(int[] prices) {  
  
        return dfs(0, true, prices);  
    }  
}
```

```

private int dfs(int i, boolean buying, int[] prices) {
    if (i >= prices.length) {
        return 0;
    }

    int cooldown = dfs(i + 1, buying, prices);
    if (buying) {
        int buy = dfs(i + 1, false, prices) - prices[i];
        return Math.max(buy, cooldown);
    } else {
        int sell = dfs(i + 2, true, prices) + prices[i];
        return Math.max(sell, cooldown);
    }
}

```

Expand

Time & Space Complexity

- Time complexity:
- $O(2^n)$
- $O(2$
- n
- $)$
- Space complexity:
- $O(n)$
- $O(n)$

2. Dynamic Programming (Top-Down)

Intuition

This problem asks for the maximum profit from buying and selling stocks, with the restriction that **after selling a stock, you must wait one day before buying again (cooldown)**.

The recursive solution tries all possible choices, but it repeats the same calculations many times. To make it efficient, we use **Dynamic Programming (Top-Down)** with memoization.

We define a state using:

- the current day `i`
- whether we are allowed to buy (`buying = true`) or must sell (`buying = false`)

For each state, we store the best profit we can achieve so that we never compute it again.

Algorithm

1. Create a memoization table `dp` where:
 - the key is `(i, buying)`
 - the value is the maximum profit from day `i` with the given state
2. Define a recursive function `dfs(i, buying)`:
 - `i` is the current day
 - `buying` indicates whether we can buy or must sell
3. If `i` is beyond the last day:
 - Return `0` since no more profit can be made
4. If the state `(i, buying)` is already in `dp`:
 - Return the stored result to avoid recomputation
5. Always consider the option to **skip the current day** (cooldown):

- Move to the next day without changing the state
- If we are allowed to buy:
 - Option 1: Buy the stock today (subtract price and move to selling state)
 - Option 2: Skip the day
 - Store the maximum of these two options in `dp`
 - If we are holding a stock:
 - Option 1: Sell the stock today (add price and skip the next day due to cooldown)
 - Option 2: Skip the day
 - Store the maximum of these two options in `dp`
 - Start the recursion from day `0` with `buying = true`
 - Return the result from this initial call

Buy and Sell Stock with Cooldown - Dynamic Programming with Memoization

Step 1 / 19

Prices

1

`0i`

2

1

3

2

0

3

2

4

DP Cache

Empty

Recursion Tree

`dfs(0,T)`

Start with prices=[1,2,3,0,2] and call dfs(0, True) to find max profit starting at day 0 in buying state.

Upgrade to Pro to unlock visualizations

0.5x1x2x

```
public class Solution {  
    private Map<String, Integer> dp = new HashMap<>();
```

```
    public int maxProfit(int[] prices) {  
        return dfs(0, true, prices);  
    }
```

```
    private int dfs(int i, boolean buying, int[] prices) {  
        if (i >= prices.length) {  
            return 0;  
        }
```

```
        String key = i + "-" + buying;  
        if (dp.containsKey(key)) {  
            return dp.get(key);  
        }
```

```
        int cooldown = dfs(i + 1, buying, prices);  
        if (buying) {  
            int buy = dfs(i + 1, false, prices) - prices[i];  
            dp.put(key, Math.max(buy, cooldown));  
        } else {  
            int sell = dfs(i + 2, true, prices) + prices[i];  
            dp.put(key, Math.max(sell, cooldown));  
        }
```

```
        return dp.get(key);  
    }
```

```
}
```

Expand

Time & Space Complexity

- Time complexity:
- $O(n)$
- $O(n)$
- Space complexity:
- $O(n)$
- $O(n)$

3. Dynamic Programming (Bottom-Up)

Intuition

This problem is about maximizing stock trading profit with a **cooldown rule**: after selling a stock, you must wait **one full day** before buying again.

Instead of using recursion, we solve this using **bottom-up dynamic programming**, where we build the solution starting from the **last day** and move backward to day 0.

At every day, we only care about two possible states:

- **buying = true** → we do not own a stock and are allowed to buy
- **buying = false** → we currently own a stock and are allowed to sell

For each day and state, we compute the **maximum profit possible from that point onward** and store it in a table.

This way, future decisions are already known when we process earlier days.

Algorithm

1. Let n be the number of days.
2. Create a 2D DP table dp of size $(n + 1) \times 2$:
 - $dp[i][1] \rightarrow$ maximum profit starting at day i when we are allowed to buy
 - $dp[i][0] \rightarrow$ maximum profit starting at day i when we are holding a stock
3. Initialize the DP table with 0 since no profit can be made after the last day.
4. Traverse days from $n - 1$ down to 0 .
5. For each day i , evaluate both states:
 - **If buying is allowed:**
 - Option 1: Buy today (subtract price and move to selling state on next day)
 - Option 2: Skip today (cooldown, stay in buying state)
 - Store the maximum of these two choices in $dp[i][1]$
 - **If holding a stock:**
 - Option 1: Sell today (add price and skip one day due to cooldown)
 - Option 2: Skip today (cooldown, stay in selling state)
 - Store the maximum of these two choices in $dp[i][0]$
6. After filling the table, the answer is stored in $dp[0][1]$, meaning:
 - starting from day 0 with permission to buy
7. Return $dp[0][1]$

Best Time to Buy and Sell Stock with Cooldown - Dynamic Programming

Step 1 / 17

Prices

1

0

2	
1	
3	
2	
0	
3	
2	
4	
DP Table	
0	
1	
0	
0	
0	
1	
0	
0	
2	
0	
0	
3	
0	
0	
4	
0	
0	
5	
0	
0	

Variables

buying→

i→

n→5

Initialize DP table with n+1 rows and 2 columns, where n=5 for prices=[1,2,3,0,2]

Upgrade to Pro to unlock visualizations

0.5x1x2x

```
public class Solution {  
    public int maxProfit(int[] prices) {  
        int n = prices.length;  
        int[][] dp = new int[n + 1][2];  
  
        for (int i = n - 1; i >= 0; i--) {  
            for (int buying = 1; buying >= 0; buying--) {  
                if (buying == 1) {  
                    int buy = dp[i + 1][0] - prices[i];  
                    int cooldown = dp[i + 1][1];  
                    dp[i][1] = Math.max(buy, cooldown);  
                } else {  
                    int sell = (i + 2 < n) ? dp[i + 2][1] + prices[i] : prices[i];  
                    int cooldown = dp[i + 1][0];  
                    dp[i][0] = Math.max(sell, cooldown);  
                }  
            }  
        }  
  
        return dp[0][1];  
    }  
}
```

Expand

Time & Space Complexity

- Time complexity:

- $O(n)$
- $O(n)$
- Space complexity:
- $O(n)$
- $O(n)$

4. Dynamic Programming (Space Optimized)

Intuition

This problem follows the same idea as the previous dynamic programming solutions:

we want to maximize profit while respecting the **cooldown rule** (after selling, we must wait one day before buying again).

In the bottom-up DP approach, we only ever use values from the **next day** and the **day after next**. That means we do not need a full DP table — we can **compress the state** into a few variables.

Instead of storing results for every day, we keep track of:

- the best profit if we are allowed to buy on the next day
- the best profit if we are allowed to sell on the next day
- the best profit if we are allowed to buy two days ahead (needed for cooldown)

By updating these values while iterating backward, we achieve the same result using constant space.

Algorithm

1. Initialize variables to represent future DP states:
 - `dp1_buy`: profit if we can buy on the next day
 - `dp1_sell`: profit if we can sell on the next day
 - `dp2_buy`: profit if we can buy two days ahead (used after selling)
2. Traverse the prices array from the last day to the first day.
3. For each day:
 - Compute the best profit if we are allowed to buy:
 - either buy today (use next day's sell profit minus price)
 - or skip today (keep next day's buy profit)
 - Compute the best profit if we are allowed to sell:
 - either sell today (use profit from two days ahead plus price)
 - or skip today (keep next day's sell profit)
4. Shift the state variables forward to represent the next iteration:
 - update `dp2_buy`
 - update `dp1_buy` and `dp1_sell`
5. After processing all days, `dp1_buy` represents the maximum profit starting from day 0 with permission to buy
6. Return `dp1_buy`

Best Time to Buy and Sell Stock with Cooldown - Dynamic Programming

Step 1 / 20

Prices

1

0

2

1

3

2

0

3

2

4i

dp1_buy

0

dp1_sell

0

dp2_buy

0

Initialize variables and start iterating from the last day (index 4) with price 3.

Upgrade to Pro to unlock visualizations

0.5x1x2x

```
public class Solution {  
    public int maxProfit(int[] prices) {  
        int n = prices.length;  
        int dp1_buy = 0, dp1_sell = 0;  
        int dp2_buy = 0;  
  
        for (int i = n - 1; i >= 0; i--) {  
            int dp_buy = Math.max(dp1_sell - prices[i], dp1_buy);  
            int dp_sell = Math.max(dp2_buy + prices[i], dp1_sell);  
            dp2_buy = dp1_buy;  
            dp1_buy = dp_buy;  
            dp1_sell = dp_sell;  
        }  
  
        return dp1_buy;  
    }  
}
```

```
}
```

Collapse

Time & Space Complexity

- Time complexity:
- $O(n)$
- $O(n)$
- Space complexity:
- $O(1)$
- $O(1)$

Common Pitfalls

Forgetting the Cooldown Day After Selling

After selling, you must skip one day before buying again. A common mistake is transitioning directly to the buying state on the next day instead of skipping to `i + 2`.

```
# Wrong: no cooldown after selling
```

```
sell = dfs(i + 1, True) + prices[i] # Should be i + 2
```

Confusing the Buying and Selling States

Mixing up which state allows buying versus selling leads to subtracting when you should add (or vice versa). When `buying=True`, you subtract the price; when `buying=False`, you add it.

Off-by-One Errors in Bottom-Up DP Bounds

When iterating backward and accessing `dp[i + 2]`, forgetting to check bounds causes index out of range errors. The DP table needs size `n + 2` or proper boundary checks.